

Sustaining 99.99% Availability for Mission-Critical Tier-One Services

A plain-English explanation of the project: what was built, why each piece exists, how a request flows through it, and what the architecture looks like.

Item	Detail
Project	Reliability engineering programme for tier-one (revenue-critical) services
Availability target	99.99% success rate, measured over a rolling 28-day window
Two halves of the work	(1) Define and measure reliability using SLIs, SLOs and error budgets. (2) Engineer the service to fail small, using timeouts, retries with exponential backoff and jitter, circuit breakers, bulkheads, request shedding, idempotency, graceful degradation and fallback workflows.
Headline outcome	The failure of any single downstream dependency no longer causes a full outage. Its impact is contained to a small, well-understood part of the user experience.
Audience	Written so that both engineers and non-technical readers can follow it end to end

Contains three full-page architecture diagrams (Figures 1, 2 and 3), each explained box by box.

Contents

If the list below appears empty, select all (Ctrl+A) and press F9 to build it. Word usually does this automatically when the document is opened.

1. The Project in Plain English

1.1 The one-sentence version

In one sentence: We kept the most important services online and correct 99.99% of the time, by first defining exactly what “working” means and measuring it honestly, and then building the service so that when something it depends on breaks, only a small, unimportant slice of the experience breaks with it — never the whole thing.

1.2 The slightly longer version

Large online systems are made of many small services that call each other. A shopping checkout, for example, might call a payments service, an inventory service, a fraud-scoring service, a recommendation service and a loyalty-points service before it can answer the customer. If any one of those is slow or broken, the naive outcome is that the whole checkout stops working. That is what most outages actually are: one small failure spreading outward until it takes down everything connected to it.

This project attacked that problem from two directions at once.

- **The measuring half.** You cannot manage what you cannot measure, and “the site feels fine” is not a measurement. We picked a small number of numbers that genuinely reflect what users experience (SLIs), set a public target for each of them (SLOs), and turned the gap between the target and perfection into a budget for failure (error budgets). That budget then drives real decisions: when it is healthy the team ships new features quickly, and when it is being spent too fast the team stops and fixes reliability instead.
- **The engineering half.** We added a standard set of protective mechanisms around every call the service makes to anything else — timeouts, careful retries, circuit breakers, bulkheads, request shedding, idempotency, fallbacks and graceful degradation. Individually each one is simple. Together they mean that a broken dependency produces a small, contained, predictable loss instead of a total outage.

The phrase for that second half is reducing the blast radius: making the damage from any single failure as small as possible.

1.3 An everyday analogy

Think about a busy hospital emergency department.

- **SLIs and SLOs** are the department's agreed measures of success — for example, “99.99% of arriving patients are seen and triaged” and “99% are triaged within 15 minutes.” Everyone agrees on the definition in advance, so nobody argues about whether a bad night was really bad.
- **The error budget** is the small, honest acknowledgement that perfection is impossible. If the target is 99.99%, then 0.01% of the time things may go wrong. That allowance is spent deliberately, not accidentally.
- **A timeout** is the rule that says a doctor waiting for a lab result does not stand in the corridor indefinitely. After a set time they proceed with what they know.

- **A retry with backoff and jitter** is calling the lab again after a short pause — and staggering those calls, so that thirty doctors do not all phone the same overloaded lab at exactly the same second.
- **A circuit breaker** is the decision to stop phoning the lab at all once you know its line is dead, and to use the backup process instead, until somebody confirms the line is back.
- **A bulkhead** is giving the cardiac team its own staff and its own rooms, so that a flood of minor injuries can never consume the resources reserved for heart attacks.
- **Request shedding** is diverting new non-urgent arrivals to a clinic when the department is genuinely full — because admitting everybody and treating nobody properly is worse for everyone.
- **Idempotency** is the wristband and patient ID system that guarantees a patient who is registered twice by accident does not get two sets of medication.
- **A fallback workflow** is the paper-based process the department switches to when the computer system is down. It is slower and less complete, but the department keeps running.
- **Graceful degradation** is the overall philosophy: when under strain, drop the non-essential activities — the routine paperwork, the elective checks — and keep the life-saving work going.

1.4 What “tier-one” and “99.99%” actually mean

Tier-one

Services are usually classified into tiers by how much damage their failure causes. A tier-one service is one where an outage directly stops revenue, breaks a legal or contractual obligation, or endangers users. Login, checkout, payments, order placement and core APIs are typical examples. Tier-one services get the strictest targets, the most engineering investment, 24/7 on-call cover and the tightest change controls. A tier-three internal reporting dashboard does not.

99.99% — “four nines”

99.99% availability means that at most 0.01% of the time, the service may be failing. That sounds abstract, so here is what it converts into as actual allowed downtime.

Target	Per year	Per 30 days	Per week	Per day
99% (two nines)	3 days 15 h	7 h 12 m	1 h 41 m	14 m 24 s
99.9% (three nines)	8 h 46 m	43 m 12 s	10 m 5 s	1 m 26 s
99.95%	4 h 23 m	21 m 36 s	5 m 2 s	43 s
99.99% (four nines)	52 m 36 s	4 m 19 s	1 m 1 s	8.6 s
99.999% (five nines)	5 m 15 s	26 s	6 s	0.9 s

Read that table carefully. Four nines allows under nine seconds of failure per day. No human can notice a problem, log in, diagnose it and fix it within nine seconds. This is the single most important consequence of the target: at 99.99%, recovery cannot depend on people. The system has to detect trouble and route around it automatically. Humans arrive afterwards to deal with the cause. Every mechanism in this document exists because of that constraint.

1.5 Availability is measured in requests, not just in minutes

Talking in minutes of downtime is a useful intuition, but the practical measure is the share of requests that succeeded. If the service handles roughly 400 requests per second, a 28-day window contains about one billion requests. A 99.99% target therefore allows about 100,000 failed requests in those four weeks.

This framing is better because real failures are rarely total. A far more common incident is “3% of requests failed for 40 minutes,” which is invisible on a simple up/down status page but is very visible in the error budget — and very visible to the affected customers.

1.6 What the finished system does differently

Situation	Before this work	After this work
A downstream service becomes slow	Threads pile up waiting for it. The pool is exhausted. Unrelated features stop responding. The whole service fails.	Each call is capped by a timeout, and each dependency has its own isolated pool. Slowness stays contained to that one feature.
A downstream service is completely down	Every request still waits for the full timeout, then fails. Latency is terrible for everyone.	The circuit breaker opens after a threshold of failures. Calls fail instantly and a fallback answer is served.
Traffic spikes far above capacity	Everything queues, everything slows, everything times out. Effective success rate approaches zero.	Excess requests are rejected quickly at the edge. The majority are served normally and correctly.
A user's payment request times out and their app retries	Risk of a double charge.	The idempotency key makes the second attempt return the first attempt's result. Exactly one charge.
A non-essential feature breaks	The page errors entirely.	The page renders without that one section. The user usually does not notice.
Deciding whether to ship a risky change	Argued case by case, usually decided by whoever is loudest.	Decided by the error budget: healthy budget means ship, exhausted budget means fix reliability first.

2. Vocabulary: Every Term Used in This Document

Keep this page open while reading the rest. Every term below is explained fully in a later chapter; these are one-line definitions to get you started.

Term	Plain-English meaning
SLI — Service Level Indicator	A number that measures one aspect of user experience, such as the percentage of requests that succeeded. It is the thermometer.
SLO — Service Level Objective	The target you commit to for an SLI, such as 99.99%. It is the temperature you are aiming for.
SLA — Service Level Agreement	A contract with customers, with financial penalties if broken. Always set looser than the internal SLO, so you find out before the customer does.
Error budget	100% minus the SLO. The amount of failure that is permitted in the window. At 99.99% over 28 days, roughly 4 minutes of total failure, or about 100,000 failed requests out of a billion.
Burn rate	How fast the error budget is being consumed compared with normal. A burn rate of 1 means you will exactly use the budget by the end of the window. A burn rate of 14.4 means it is gone in about two days.
Blast radius	How much of the system, and how many users, are affected by one failure. The core aim of this project is to shrink it.
Tier-one service	A service whose failure immediately costs revenue, breaks a legal obligation, or endangers users.
Dependency	Anything the service needs in order to answer: another service, a database, a cache, a queue, an external API.
Latency	How long a request takes to answer.
p99 latency	The value that 99% of requests come in under. If p99 is 300 ms, then 99 of every 100 requests were faster than 300 ms and one was slower. Averages hide pain; percentiles reveal it.
Timeout	The maximum time you are willing to wait for something before giving up.
Deadline	A time budget for the whole request, passed along to every hop, so the entire chain knows when it is pointless to continue.
Retry	Trying a failed call again, in the hope that the failure was temporary.
Exponential backoff	Waiting longer between each successive retry — for example 100 ms, then 200 ms, then 400 ms — instead of hammering immediately.

Term	Plain-English meaning
Jitter	Adding a random amount to each wait, so that many clients do not all retry at exactly the same instant.
Thundering herd	The stampede that happens when many clients act in unison, for example all retrying at once, or all rebuilding the same expired cache entry.
Retry storm	Retries multiplying the load on a service that is already struggling, which makes it struggle more — a self-reinforcing failure.
Circuit breaker	A switch that trips open after repeated failures, so calls to a known-broken dependency fail instantly instead of waiting.
Bulkhead	Separating resources so that one dependency's problem cannot consume the resources needed by everything else. Named after the watertight compartments in a ship.
Request shedding	Deliberately rejecting some requests quickly when overloaded, so that the rest can be served properly.
Idempotency	The property that performing an operation twice has exactly the same effect as performing it once. It is what makes retrying a payment safe.
Idempotency key	A unique identifier the client attaches to a write, so the server can recognise a repeat of the same logical operation.
Fallback	A cheaper, simpler answer used when the ideal answer is unavailable — a cached value, a default, or a reduced version of the feature.
Graceful degradation	Deliberately reducing functionality under stress so the core function survives, instead of failing completely.
Cascading failure	One component's failure overloading the next, which overloads the next, until the whole system is down.
Fail fast	Returning an error immediately rather than making the caller wait. Counter-intuitively, this usually improves overall availability.
Cell / shard	An isolated copy of the stack serving a subset of users, so a bad failure hits only that subset.
Canary release	Sending a new version a tiny share of traffic first, watching the SLIs, and only then rolling it out widely.

3. Part One — Measuring Reliability: SLIs, SLOs and Error Budgets

This chapter covers the half of the project that defines and measures reliability. Without it, the engineering work in Chapter 4 would have no target, no way of knowing whether it worked, and no way of deciding how much more to invest.

3.1 SLI — the measurement

What it is

A Service Level Indicator is a single number, expressed as a ratio, that describes one dimension of the experience the service is delivering. The standard shape is:

$$\text{SLI} = (\text{number of good events} \div \text{number of valid events}) \times 100\%$$

Both words in that formula matter.

- **Good** must be defined precisely and in advance. For an availability SLI, “good” usually means the response was not a server error (not a 5xx) and it arrived within a stated time. A request that succeeds after nine seconds is not good; the user has already given up.
- **Valid** means the events you are willing to be judged on. Automated health checks, load-test traffic and requests that were malformed by the client (400-class errors) are excluded, because counting them would distort the picture in either direction.

Where the measurement is taken

This detail matters more than almost anything else in the chapter. A service measuring itself will always report better numbers than reality, because it cannot record the requests that never reached it — the ones lost to a failed load balancer, an overloaded queue, or a network problem. The SLI is therefore measured as close to the user as possible: at the edge load balancer, in the client itself, or from external probes. The guiding question is always “what did the user actually experience?”, not “what did the server believe it did?”

The categories of SLI used

Type of SLI	What it measures	Typical definition used
Availability	Did the request work at all?	Non-5xx responses ÷ all valid responses, measured at the edge
Latency	Was it fast enough to be useful?	Share of requests answered in under 300 ms (p99 tracked separately)
Correctness / quality	Was the answer right?	Share of responses that passed an automated correctness check, or that were served fully rather than in degraded form

Type of SLI	What it measures	Typical definition used
Freshness	Was the data recent enough?	Share of reads served with data less than 60 seconds old
Durability	Did we lose anything we accepted?	Share of accepted writes that were still retrievable afterwards
Coverage	Did the batch or pipeline process everything?	Share of records processed successfully within the expected window

Rule of thumb: Between two and four SLIs per service is right. One is not enough to describe the experience; ten means nobody looks at any of them. Choose the ones whose movement would genuinely make a user unhappy.

3.2 SLO — the target

What it is

A Service Level Objective is the target value for an SLI, over a stated window. “99.99% of valid checkout requests succeed within 300 ms, measured over a rolling 28-day window” is a complete SLO. It is an internal commitment made by the team that owns the service.

A complete SLO specification

A target on its own is ambiguous, so each SLO was written down in full. This is the actual template used:

Field	Example value	Why it is needed
Name	Checkout availability	So it can be referenced in reviews and incidents
SLI definition	Proportion of valid POST /checkout requests returning a non-5xx status within 300 ms	Removes all ambiguity about what counts as good
Measurement point	Edge load balancer logs	Prevents the service from grading its own homework
Target	99.99%	The commitment
Window	Rolling 28 days	Rolling, not calendar, so the budget never magically resets on the first of the month
Exclusions	Client 4xx errors, synthetic health checks, traffic during announced maintenance windows	Keeps the number honest in both directions
Owner	The team that owns the service	Someone must be accountable

Field	Example value	Why it is needed
Consequence	If the budget is exhausted, feature releases pause until it recovers	An SLO with no consequence is decoration

Why the target is not 100%

Choosing 100% would be a mistake, not an ambition, for four reasons.

1. It is impossible. Networks drop packets, hardware fails, cables get cut and dependencies have their own limits. A service that calls three dependencies each promising 99.99% cannot itself exceed roughly 99.97% without adding fallbacks.
2. It is unaffordable. Each additional nine typically multiplies cost — more redundancy, more regions, more review, more testing. The money is better spent elsewhere once users stop noticing the difference.
3. It would stop all change. The overwhelming majority of incidents are caused by a change: a deployment, a configuration edit, a schema migration. The only way to approach 100% is to stop changing anything, which slowly kills the product.
4. Users cannot perceive it. A mobile user on a patchy network already experiences worse than four nines from their own connection. Perfecting the server side beyond that point delivers no experience the user can detect.

So the target is set at the level where users are reliably happy and the business can afford it — and the remaining sliver of failure is treated as a resource to be spent wisely.

3.3 Error budget — the allowance

What it is

The error budget is simply what is left over from the target:

Error budget = 100% – SLO. At an SLO of 99.99%, the budget is 0.01% of all valid requests in the window — approximately 4 minutes of total unavailability, or about 100,000 failed requests out of a billion, in each rolling 28-day period.

The idea that changes behaviour is this: the budget is not a warning line to be avoided. It is a resource, and it is there to be spent. Spending it on a canary release that had to be rolled back, or on a risky but valuable migration, is a perfectly good use of it. Leaving it entirely unspent every month is itself a signal — it suggests the team is being too cautious and could be shipping faster, or that the target was set too low.

The budget policy

The policy is agreed in advance, in writing, while everyone is calm. That is the entire point: during an incident nobody has to argue about whether reliability or features come first, because the answer was decided beforehand.

Budget remaining	What it means	Agreed action
Above 75%	Comfortably reliable	Ship normally. Consider taking more risk: bigger changes, faster rollouts, deliberate failure testing.
50–75%	Normal	Ship normally. Standard canary and rollout process.
25–50%	Warning	Slow down. Smaller changes, longer canary bake times, no risky migrations on Fridays.
Below 25%	Serious	Feature work pauses for anything not reliability-related. The top reliability items move to the front of the queue.
Exhausted (0%)	Target missed	Full change freeze except for fixes that improve reliability. A written review of the causes, and an agreed plan, before the freeze lifts.

3.4 Burn rate — knowing early, and not being woken up for nothing

Waiting for the budget to be fully spent before reacting is far too late. Instead the team alerts on the burn rate: how fast the budget is being consumed relative to normal.

A burn rate of 1 means the budget will be exactly used up at the end of the 28-day window — that is the definition of on-target. A burn rate of 10 means it is being spent ten times too fast, and will be gone in under three days.

Burn rate	Budget gone in	Detection windows	Response
14.4×	About 2 days	Confirmed over 1 hour, cross-checked over 5 minutes	Page a human immediately, day or night
6×	About 4.5 days	Confirmed over 6 hours, cross-checked over 30 minutes	Page a human
3×	About 9 days	Confirmed over 24 hours, cross-checked over 2 hours	Open a ticket for the next working day
1×	Exactly 28 days	Trend view only	No alert; this is the expected rate

Two details make this scheme work in practice. First, every alert uses two windows at once — a long one to confirm the problem is real and a short one to confirm it is still happening — which stops both false alarms and alerts that keep firing long after the incident is over. Second, the alerts are on symptoms the user feels (requests failing, requests slow), not on causes such as high CPU. A machine with high CPU that is serving every request correctly is not an emergency and nobody should be woken for it.

3.5 Why this half of the project changes team behaviour

- **It ends the reliability-versus-features argument.** Both sides get a number instead of an opinion, and the number decides.
- **It makes risk explicit.** “This migration might cost us 15% of the budget” is a sentence a product manager can weigh up.
- **It makes alerts trustworthy.** Because alerts fire on user-visible symptoms tied to the budget, an engineer who is paged knows it is real. Trust in the pager is what makes fast response possible.
- **It gives incidents an agreed cost.** Post-incident, the question is not “was that bad?” but “that cost 22% of the quarter's budget — what will we change so it does not happen again?”

4. Part Two — The Protective Components, One by One

This chapter explains every mechanism named in the project, in the order in which it makes sense to learn them. Each one follows the same structure: what it is in a line, the problem it solves, how it works step by step, the settings that matter, and what goes wrong if it is done badly.

The single idea underneath all of them: A tier-one service is only as available as the way it fails. You cannot prevent dependencies from breaking. You can decide, in advance, exactly how much of your service breaks with them.

4.1 Graceful degradation — the governing philosophy

In one line: When something breaks, drop the least important parts of the experience and keep the essential parts working, instead of failing completely.

The problem it solves

Most software is written on an unspoken assumption: every call succeeds, and if one does not, the whole operation fails. A product page that needs the recommendation service will, by default, return an error page when recommendations are unavailable — even though the price, the photographs, the description and the “Add to basket” button were all perfectly available. The user is denied a working page because one decorative section could not be built. That is an enormous, entirely self-inflicted loss.

How it works, step by step

5. Classify every dependency and every feature by criticality. This is a business decision, not a technical one, and it must be written down. Typically three classes: critical (the request is meaningless without it), important (the experience is noticeably worse without it) and optional (most users would not notice).
6. Define the behaviour for each class when its dependency fails. Critical: fail the request honestly and quickly with a clear message. Important: substitute a cached or default answer. Optional: omit the section entirely and render everything else.
7. Define named degradation levels for the whole service, so that under severe stress the system can step down in a predictable, tested order rather than improvising.
8. Trigger the levels automatically from signals — error rates, latency, circuit-breaker state, queue depth — and also expose a manual override switch that on-call engineers can flip during an incident.
9. Make the current level visible: emit it as a metric, show it on the dashboard, and record it in the response so that traces and logs show which requests were degraded.
10. Recover automatically. When the signals return to normal, step back up. Degradation that requires a human to undo it becomes a permanent, forgotten reduction in quality.

The degradation levels used

Level	Name	What the user gets	Typically triggered by
L0	Full service	Everything: personalised results, live inventory, reviews, recommendations	Normal operation
L1	Personalisation off	Everything, but with generic instead of personalised content	The recommendation or profile dependency is failing
L2	Cached reads	Slightly stale prices and stock levels, clearly labelled	The primary database is slow or overloaded
L3	Read-only	Browsing and viewing work; new writes are queued and confirmed later	The write path or primary database is unavailable
L4	Core only	Only the critical journey (for example, checkout for existing baskets); everything else is disabled to protect capacity	Severe overload or a major dependency outage
L5	Static holding page	An honest status message with an estimate, served from the CDN	Total failure of a critical dependency

Non-obvious but essential: Every fallback path must be exercised regularly — in practice by deliberately breaking dependencies in controlled game-day exercises. A fallback that has never run in production is not a fallback; it is untested code that will be executed for the first time during your worst incident.

4.2 Timeouts — never wait forever

In one line: Every single call to anything external has a hard maximum time it is allowed to take, after which the caller gives up and moves on.

The problem it solves

A server can handle a fixed number of requests at once, because each in-flight request occupies a thread, a connection and some memory. When a dependency becomes slow, requests waiting on it are not free — they hold those resources the entire time. If the dependency takes 30 seconds instead of 50 milliseconds, each request occupies its slot 600 times longer. Within moments every slot is occupied by a request that is waiting, and the service can no longer accept anything at all, including requests that have nothing to do with the slow dependency.

The key insight: A dependency that is slow is more dangerous than a dependency that is down. A failure returns immediately and releases the resources; slowness silently consumes the entire service's capacity while looking healthy on a simple up/down check. Timeouts are what convert dangerous slowness into safe, fast failure.

The layers of timeout, and why each is needed

Timeout	What it limits	Typical value	Why it exists separately
Connection	Establishing the TCP or TLS connection	50–200 ms	A host that is unreachable should be detected almost instantly, not after the full read timeout
Read / socket	Waiting for bytes to arrive once connected	Set from the dependency's p99.9	This is the one people forget; many client libraries default it to infinity
Per-attempt	One complete attempt including connect and read	e.g. 250 ms	This is the unit that a retry repeats
Total call	All attempts plus the waits between them	e.g. 550 ms	Stops the retry logic from quietly tripling the time budget
Request deadline	The whole user request, end to end, across every hop	e.g. 800 ms	The overall promise to the user; everything else must fit inside it
Idle / pool	How long an unused connection is kept	Seconds to minutes	Prevents stale connections to hosts that have silently disappeared

Deadline propagation — the part that ties it together

A timeout on a single call is not enough on its own, because a chain of calls can still overrun. The technique is to give the whole request one deadline at the front door and then carry the remaining time with it through every hop, in a header.

11. The gateway stamps the request with a total budget, say 800 ms, and records the arrival time.
12. Before each downstream call, the service calculates the remaining budget: the original deadline minus the time already elapsed.
13. The effective timeout for that call is the smaller of its configured timeout and the remaining budget. There is no point allowing a 250 ms call when only 90 ms of the promise remains.
14. The remaining budget is passed onward in the request headers, so the next service down the chain applies the same logic.
15. If the remaining budget is already too small to do useful work, the call is abandoned immediately and the fallback is used. This avoids spending real capacity producing an answer that will arrive after the caller has stopped listening.

The budget arithmetic that must always hold: The sum of every hop's total call timeout, including its retries and backoff waits, must be less than the caller's deadline. If the inner timeouts are larger than the outer deadline, the outer caller gives up first and every resource spent by the inner services is wasted — while their dashboards still show them working hard. This is one of the most common causes of an outage that nobody can explain.

How the values are chosen

- **Start from the dependency's real latency distribution, not from a guess.** A common choice is roughly the p99.9 latency plus a margin. If the dependency answers 99.9% of calls within 180 ms, a 250 ms timeout is sensible.
- **Too short is not safe.** It fails requests that would have succeeded, and each of those failures generates a retry, which adds load, which increases latency, which trips more timeouts. This is a well-known way to turn a small slowdown into a full outage.
- **Too long is not safe either.** It is the resource-exhaustion scenario above.
- **Review after every latency change.** Timeouts are configuration that ages. A dependency that got slower over six months will silently start tripping timeouts that were correct when they were set.

4.3 Retries with exponential backoff and jitter

In one line: When a call fails in a way that might be temporary, try again — but only for operations that are safe to repeat, waiting longer after each attempt, with a random element so that everybody does not retry in unison.

The problem it solves, and the problem it creates

Many failures in distributed systems are genuinely transient: a brief network blip, a leader election, one instance restarting mid-deployment. Retrying converts those into invisible successes and is one of the cheapest availability wins available.

But retries are also one of the most reliable ways to turn a small problem into an outage. If a dependency starts failing and every client immediately retries three times, that dependency now receives three to four times its normal traffic at precisely the moment it is least able to cope. It gets slower, more calls fail, more retries are generated, and the system spirals. This is a retry storm, and it is why the details below are not optional refinements — they are the difference between a retry policy that helps and one that causes the incident.

The three pieces of the technique

Piece 1 — Exponential backoff

Instead of retrying immediately, wait a period that doubles with each attempt: 100 ms, then 200 ms, then 400 ms, up to a cap. This gives the struggling dependency progressively more room to recover, and it means that a client which keeps failing applies steadily less pressure rather than more.

Piece 2 — Jitter

Backoff alone is not enough. Consider a dependency that fails for two seconds, during which a thousand clients each get an error. With pure exponential backoff, all thousand wait exactly 100 ms and then retry at the same instant, producing a spike far worse than normal traffic. Then they all wait exactly 200 ms and do it again. The load arrives in synchronised waves that repeatedly knock over the recovering service.

Jitter breaks the synchronisation by randomising each wait. The common variants are:

Variant	Wait calculation	Behaviour
No jitter	$\text{delay} = \text{base} \times 2^n$	Clients stay perfectly synchronised. Avoid.
Full jitter	$\text{delay} = \text{random between } 0 \text{ and } (\text{base} \times 2^n)$	Spreads retries most evenly. The usual default choice.
Equal jitter	$\text{delay} = \text{half the backoff, plus a random amount up to the other half}$	Spreads load while guaranteeing a minimum wait.
Decorrelated jitter	$\text{delay} = \text{random between base and } (\text{previous delay} \times 3), \text{ capped}$	Adapts well to long outages; used in some AWS client libraries.

Piece 3 — Strict rules about when to retry at all

Situation	Retry?	Reason
Timeout, connection reset, 503 Service Unavailable	Yes	Classic transient conditions that often succeed on a second attempt
429 Too Many Requests with a Retry-After header	Yes, after the stated delay	The server has told you exactly when to come back; ignoring it makes things worse
500 Internal Server Error	Usually once, cautiously	May be transient, but may also be a deterministic bug that no number of retries will fix
400, 401, 403, 404, 422 — client errors	No	The request is wrong. Repeating it wastes capacity and will fail identically every time.
A non-idempotent write with no idempotency key	No	The first attempt may well have succeeded before the response was lost. Retrying risks a duplicate charge or a duplicate order.
The remaining request deadline is nearly spent	No	The answer would arrive after the caller has given up
The circuit breaker for that dependency is open	No	The dependency is known to be down; go straight to the fallback

The two safeguards that are most often missed

- **A retry budget.** Cap retries as a proportion of total traffic — for example, retries may never exceed 10% of requests to a given dependency, enforced with a token bucket. When a dependency is broadly failing, retrying is pointless anyway; the budget guarantees that a mass failure cannot multiply into a flood. This single control prevents most retry storms.
- **Retrying at only one layer.** If the mobile client retries three times, the gateway retries three times and the service retries three times, one user action becomes twenty-seven calls to the dependency. Decide which single layer owns retries for each hop — usually the layer closest to the failure — and make every other layer pass the failure through untouched.

A worked example

Base delay 100 ms, three attempts maximum, full jitter, per-attempt timeout 250 ms, and a total call budget of 550 ms:

Attempt	Wait before it	Worst case elapsed	Notes
1	0 ms	250 ms	The original call
2	Random 0–100 ms	600 ms	Only proceeds if the error was retryable and budget remains

Attempt	Wait before it	Worst case elapsed	Notes
3	Random 0–200 ms	1,050 ms	Would exceed the 550 ms call budget, so in practice it is skipped and the fallback is used

That last row is deliberate. The policy allows three attempts, but the deadline is the ultimate authority: the third attempt only happens if there is genuinely time for it. The budget always wins over the retry count.

4.4 Circuit breakers — stop calling what is already broken

In one line: After a dependency fails repeatedly, stop calling it entirely for a while; fail instantly and use the fallback, then test cautiously to see whether it has recovered.

The problem it solves

Timeouts stop a single call from hanging, but they do not stop you from making pointless calls. If a dependency is completely down, every request still waits the full timeout before failing. At 250 ms per call with a thousand requests per second, the service is permanently holding 250 concurrent slots for calls that are guaranteed to fail. Users wait 250 ms longer than necessary, every time, to receive the same answer they would have got instantly. Meanwhile the failing dependency is being hammered with traffic it cannot serve, which prevents it from recovering.

The three states

State	What happens to calls	How it is entered	How it is left
CLOSED (normal)	All calls pass through to the dependency as usual, while failures are counted in a rolling window	The default state, and the state returned to after a successful recovery	Moves to OPEN when the failure rate in the window crosses the threshold and the minimum call volume has been met
OPEN (tripped)	No calls are attempted. Every request fails immediately and the fallback path runs.	Entered when the failure threshold is breached	After the cool-off period expires, moves to HALF-OPEN
HALF-OPEN (testing)	A small number of trial calls are allowed through; all others still fail fast	Entered automatically when the cool-off timer ends	Back to CLOSED if the trial calls succeed; straight back to OPEN if any of them fail

The half-open state is what makes the pattern safe. Without it, a breaker either stays open forever or slams the full traffic load back onto a dependency the instant the timer expires, knocking it over again. Allowing a handful of trial calls tests recovery at a load the dependency can survive.

The settings that matter

Setting	Typical value	What it controls
Rolling window	10–60 seconds, or the last N calls	How much history counts toward the decision
Minimum call volume	20 calls in the window	Prevents tripping on a tiny sample — two failures out of two is not evidence
Failure-rate threshold	50%	How bad it has to get before opening

Setting	Typical value	What it controls
Slow-call threshold	e.g. 50% of calls slower than 200 ms	Lets the breaker react to a dependency that is crippling but not technically failing
Open duration (cool-off)	10–60 seconds	How long to leave it alone before testing again
Half-open trial calls	3–10	How much evidence of recovery is required
Scope	One breaker per dependency, per operation	A failing search endpoint should not stop calls to a healthy write endpoint on the same host

What good looks like

- The breaker's state is a first-class metric and appears on the dashboard. An engineer should be able to see at a glance which dependencies are currently cut off.
- Opening a breaker is an event worth investigating, but it is not automatically an emergency — that is the entire point. The system has already protected itself; the alert can be proportionate.
- Breakers protect calls to a critical dependency too. There may be no useful fallback for payments, but failing in 1 ms with a clear message is still much better for the user, and much cheaper for the service, than failing in 250 ms.
- Breakers are scoped per instance where possible, so that one sick replica behind a load balancer is avoided while the healthy replicas continue to be used.

4.5 Bulkheads — isolate resources so one failure cannot drain everything

In one line: Give each dependency its own limited allocation of threads and connections, so that a problem with one can never consume the resources the others need.

The problem it solves

The name comes from shipbuilding. A ship's hull is divided into sealed compartments; if one is breached, that compartment floods and the ship stays afloat. Without the bulkheads, a single hole sinks the vessel.

Software has the same weakness. Most services use one shared pool of worker threads for everything. If the recommendation service becomes slow, requests waiting on it accumulate in that shared pool. Once the pool is full, no request of any kind can be served — including checkout, which does not use recommendations at all. A cosmetic dependency has taken down the revenue path purely because they shared a resource pool.

How it works, step by step

- 16. Identify every distinct downstream dependency the service calls.
- 17. Give each one its own bounded pool: a dedicated thread pool, or more cheaply a semaphore that caps the number of concurrent calls to that dependency.
- 18. Size each pool from measured behaviour rather than intuition (see the calculation below).
- 19. When a pool is full, do not queue indefinitely. Reject immediately and go to the fallback — a full bulkhead behaves exactly like an open circuit breaker.
- 20. Export the utilisation of every pool as a metric. A pool that is regularly near its limit is an early warning that arrives long before user-visible failure.

Sizing a pool

The starting point is Little's Law, which in plain terms says: the number of things in progress equals the rate of arrivals multiplied by how long each one takes.

Worked example: A dependency receives 200 calls per second and answers in 50 ms on average. $\text{Concurrency} = 200 \times 0.05 = 10$. So around 10 calls are in flight at any moment under normal conditions. Sizing the pool at 15–20 gives headroom for spikes, while still capping the damage: even if that dependency hangs completely, it can never tie up more than 20 workers.

Levels of isolation, from cheap to strong

Level	What is separated	Protects against	Cost
Semaphore limit	Concurrent calls per dependency, on the caller's own threads	One slow dependency consuming all workers	Almost none

Level	What is separated	Protects against	Cost
Dedicated thread pool	Threads as well as concurrency	The above, plus blocking behaviour inside the client library	Some memory and context-switching
Separate connection pools	Sockets and database connections per dependency	One dependency exhausting the shared connection limit	Low
Separate instance groups	Whole servers per traffic class — for example, one group for interactive traffic, another for batch	Bulk or background work starving interactive users	Extra capacity
Cells and shuffle sharding	Independent copies of the stack, each serving a subset of customers	Any single failure affecting all customers at once	Significant, but the strongest containment available

Shuffle sharding deserves a brief note because it is unusually effective. If customers are assigned to a random pair of shards rather than a single one, then even a customer whose traffic destroys both of their shards overlaps completely with only a tiny fraction of other customers. With eight shards and pairs, one “poisonous” customer fully affects roughly one in twenty-eight of the others, instead of everyone.

4.6 Request shedding — serve most people well rather than everybody badly

In one line: When more work arrives than the service can possibly do, reject the excess immediately and cheaply, so that the requests you do accept are actually served correctly and quickly.

The problem it solves

Every service has a maximum throughput. The instinctive response to exceeding it is to queue the surplus. That instinct is wrong, and understanding why is the key to this section.

When arrivals exceed capacity, the queue does not absorb the excess — it grows without bound. Every request in the queue waits longer. Eventually all of them wait longer than their client's timeout. At that point the service is working at full capacity, consuming CPU, memory and database connections, producing answers that every single client has already given up on. Throughput of useful work collapses toward zero at the exact moment the servers look busiest. This is congestion collapse.

The trade-off, stated plainly: At 130% of capacity you can either serve 0% of users successfully by trying to serve everyone, or serve roughly 77% of them perfectly and turn the rest away in a few milliseconds with a clear 'try again shortly'. The second outcome is better for every measure that matters, including the error budget — and a fast, honest rejection lets a well-behaved client retry successfully a moment later.

How it works, step by step

21. Decide the signal that means 'overloaded'. Queue wait time is the best single signal, because it directly reflects the delay users are experiencing. Concurrency limits, CPU saturation and queue depth are also used.
22. Check that signal at admission — before any expensive work is done. Shedding must be far cheaper than serving, or the shedding itself becomes the overload.
23. Choose what to shed by priority, never at random. Background and batch traffic is shed first, then retried traffic, then anonymous browsing, and the critical journey (checkout, payment, login) is protected last.
24. Reject with a clear signal: HTTP 429 or 503, plus a Retry-After header so that clients back off in an orderly way rather than immediately hammering again.
25. Drop requests that are already past their deadline without processing them at all. Work on a dead request is pure waste — nobody is listening for the answer.
26. Prefer a last-in-first-out queue during overload. This sounds unfair, but it means the requests being served are the freshest ones with time left on the clock, rather than a queue of stale requests that will all time out anyway.

Where shedding is applied, and why in several places

Layer	Mechanism	What it protects
CDN and edge	Bot filtering, global rate limits, static caching	Stops floods and abuse before they consume any application capacity
API gateway	Per-customer and per-API-key quotas	Fairness — one noisy client cannot starve everyone else
Service admission control	Adaptive concurrency limits, queue-age checks, priority-based dropping	The service's own ability to keep its latency promises
Database and cache tier	Connection limits, statement timeouts, admission queues	The shared data layer, which is usually the hardest thing to scale quickly

An adaptive concurrency limit is worth calling out. Rather than configuring a fixed request-per-second cap that ages badly, the service continuously measures its own latency and adjusts its concurrency limit up or down — the same principle that TCP uses to find the right sending rate. It discovers real capacity automatically as hardware, traffic mix and code change.

4.7 Idempotency — what makes retrying safe

In one line: An operation is idempotent if doing it twice produces exactly the same result as doing it once — which is what allows a client to retry a payment without any risk of being charged twice.

The problem it solves

Consider the most common failure in any distributed system: a client sends a payment request, the server processes it successfully, and then the response is lost on the way back. The client sees a timeout.

The client now faces a genuinely impossible question: did that work or not? It cannot tell the difference between ‘the request never arrived’ and ‘the request succeeded but the answer was lost.’ If it retries, it might double-charge the customer. If it does not retry, it might have silently lost a payment the customer expected to be made.

Idempotency removes the dilemma entirely. It makes retrying always safe, so the client can simply retry.

How it works, step by step

- 27. The client generates a unique idempotency key — typically a UUID — for each logical operation, not for each network attempt. All retries of the same logical operation carry the same key.
- 28. The key is sent with the request, usually in an Idempotency-Key header.
- 29. The server, before doing any work, atomically records the key as in-progress in a fast store such as Redis or a database table with a unique constraint. If the key already exists, this insert fails, and that failure is the duplicate detection.
- 30. If the key is new, the operation is performed. Crucially, the completed status and the exact response body are saved against the key in the same transaction as the business change, so the two can never disagree.
- 31. If the key already exists and is marked complete, the stored response is returned immediately. No work is repeated. The client cannot tell the difference from the original, which is exactly what is wanted.
- 32. If the key already exists and is still in progress, the server either waits briefly for the first attempt to finish or returns a status telling the client to retry shortly. It must never start a second execution.
- 33. A fingerprint (a hash) of the request body is stored with the key. If the same key arrives with a different body, that is a client bug, and the request is rejected with an error rather than silently returning the wrong stored answer.
- 34. Keys expire after a defined retention period — 24 hours is a common choice, comfortably longer than any client will keep retrying.

Which operations are naturally safe

Operation	Naturally idempotent?	What is needed
GET, HEAD, OPTIONS	Yes — they change nothing	Safe to retry freely

Operation	Naturally idempotent?	What is needed
PUT (replace with a full value)	Yes — setting a value twice gives the same value	Safe, provided it truly replaces rather than appends
DELETE	Yes in effect — the resource ends up gone either way	Decide whether a second delete returns 204 or 404, and be consistent
POST (create a new thing)	No — two calls create two things	Requires an explicit idempotency key. This is the important case.
'Add 50 to the balance'	No — running it twice adds 100	Rewrite as a conditional set, or protect it with a key
'Set the balance to 150 if the version is 7'	Yes — the second attempt finds version 8 and does nothing	Optimistic concurrency using a version number
Publishing to a message queue	No — most queues deliver at least once	The consumer must deduplicate on a message ID before acting

Why this is placed before retries in the architecture: Retries are only safe because idempotency exists. If the idempotency layer is broken or bypassed, the retry policy stops being a reliability feature and becomes a mechanism for duplicating customer orders and payments. The two must always be designed together.

4.8 Fallback workflows — always have a plan B

In one line: For every dependency, decide in advance what answer to give when it is unavailable — and make sure that answer is cheap, pre-prepared and regularly tested.

The problem it solves

Timeouts, breakers and bulkheads all end the same way: the call did not produce an answer. Something still has to be returned to the user. Without a planned fallback, that something is an error page — which means all the protective machinery has succeeded only in failing faster. The fallback is what converts fast failure into continued service.

The fallback hierarchy

Fallbacks are arranged in order of preference. The system walks down this ladder until it finds something it can deliver.

Rank	Fallback	Example	What the user notices
1	Fresh data from the primary source	A live call to the inventory service	Nothing — this is normal operation
2	A second source or a read replica	Read from the replica when the primary is unavailable	Nothing, or data a few seconds old
3	Recent cached data	The price from the cache, up to 60 seconds old	Nothing, in almost all cases
4	Deliberately stale cached data	Yesterday's cached recommendations	Slightly less relevant content
5	A static or computed default	The generic best-sellers list; the standard shipping estimate	Generic instead of personalised content
6	Omit the feature	The page renders with no reviews section	A missing section, often unnoticed
7	Accept now, complete later	The order is queued and confirmed by email within minutes	A short delay and a clear message
8	Fail honestly and quickly	'Payments are temporarily unavailable. Nothing has been charged. Please try again in a few minutes.'	A clear, trustworthy error rather than a spinner

The rules that make fallbacks trustworthy

- **A fallback must never depend on the thing that failed.** A cache that can only be filled by calling the broken service is not a fallback. It must be filled continuously in advance, so that it is already warm at the moment it is needed.

- **A fallback must be cheap.** It runs precisely when the system is under the most stress. An expensive fallback — a heavy database query, a complex recalculation — becomes a second outage.
- **A fallback must be tested in production.** Regular game days, where a dependency is deliberately disabled under supervision, are the only way to know the path works.
- **A fallback must be measured.** Emit a metric every time one is used. Otherwise the service can run in a permanently degraded state for weeks while every dashboard shows a healthy 100% success rate. This is one of the most common failure modes of an otherwise good design.
- **Choose fail-open or fail-closed deliberately.** If the recommendation service is down, fail open and show generic content. If the authorisation or fraud service is down, fail closed and deny access — an available service that lets everybody in is far worse than a brief outage. This must be an explicit, documented decision for every dependency.
- **Be honest with the user.** Stale data that is silently presented as live destroys trust. A small, clear note — ‘prices updated a few minutes ago’ — keeps the degraded experience trustworthy.

4.9 Blast radius — the outcome all of this is aiming at

Blast radius is how far the damage from one failure spreads: how many users, how many features and how many other services are affected. Every component in this chapter shrinks it along a different axis.

Component	The dimension of damage it bounds
Timeouts	How long a single failure can hold on to a resource
Bulkheads	How many resources one dependency is ever allowed to hold
Circuit breakers	How many doomed calls are attempted at all
Retry limits, budgets and jitter	How much extra load a failure is allowed to generate
Request shedding	How many users are affected when demand exceeds capacity
Idempotency	Whether a retry can cause permanent, incorrect data changes
Fallbacks and graceful degradation	How much of the user experience is lost when something is missing
Cells, shards and canary releases	How many customers a single bad instance or release can reach

5. The Architecture Diagram

Figure 1, on the following page, shows the whole system at once. Read it top to bottom: a request enters at the top and travels downward through six layers, each of which removes a class of danger before the request reaches the next. The column on the right is the measurement loop from Chapter 3, which runs continuously alongside everything else. The rest of this chapter explains every box in turn.

5.1 How to read the diagram

Before the walkthrough, here is what the visual language in Figure 1 means.

Visual element	Meaning
Dashed outer panels	A layer of the system. Everything inside a panel sits at the same stage of the journey.
Solid arrows pointing down	The normal path of a request when everything is healthy.
Dashed arrows	Paths taken only in unusual conditions: fallbacks, telemetry flows and feedback.
Red outlined tags	A named resilience pattern, positioned at the exact point in the stack where it is applied.
Green boxes	Critical dependencies — the request cannot succeed without them.
Grey boxes	Optional dependencies — the request can succeed without them, in reduced form.
Amber boxes	Protective or fallback machinery rather than business logic.
The right-hand column	The measurement and control plane. It does not serve user traffic; it observes it and drives decisions.

Every request travels downward through a stack of guards. The right-hand column is the measurement loop that proves the 99.99% target is actually being met.



5.3 The request path, layer by layer

Layer 1 — Clients: who calls the service

The three boxes represent the kinds of caller the service must cope with. They are drawn because the client is part of the reliability design, not merely a consumer of it.

- **Mobile app.** Retries on failure using exponential backoff and jitter. Mobile clients matter most here because they run on unreliable networks and therefore retry most often. A badly written mobile retry loop can generate a self-inflicted denial-of-service attack on your own service.
- **Web browser.** Attaches an idempotency key to any write, so that a user who double-clicks 'Pay' or refreshes during a slow response does not create two orders.
- **Partner or business API.** Machine traffic, which is high-volume and utterly indifferent to whether the service is struggling. It is bound by a quota for exactly that reason.

Layer 2 — Edge: the first line of defence

The edge exists to absorb pressure before it reaches anything expensive.

- **CDN and WAF.** The content delivery network serves cached and static content without the service being involved at all — the cheapest possible form of protection, since traffic that never arrives cannot overload anything. The web application firewall blocks abusive and malicious traffic.
- **Global load balancer.** Spreads traffic across healthy instances and regions, and removes failing instances within seconds based on health checks. This is what makes a single machine's death a non-event.
- **Rate limiting and request shedding.** The first shedding point. Traffic beyond what the system can serve is rejected here with an HTTP 429 and a Retry-After header. Rejection at the edge costs almost nothing, which is precisely why it belongs at the edge — the further into the system a request travels before being rejected, the more capacity its rejection has already wasted.

Layer 3 — API gateway: identity, budget and fairness

- **Authentication and authorisation.** Establishes who the caller is and whether they are permitted to do this. Placed before the expensive work so that rejected requests cost as little as possible.
- **Deadline stamp.** This box is easy to overlook and is one of the most important in the diagram. The gateway assigns the request a total time budget — for example 800 ms — and that deadline then travels with the request through every subsequent hop. Every timeout deeper in the system is bounded by whatever remains of it.
- **Per-tenant quota.** Fairness. Limits are applied per customer or per API key, so that one client behaving badly consumes only its own allowance rather than the service's entire capacity.
- **Routing.** Sends the request to the correct service and version. This is also where canary releases are steered, by sending a small percentage of traffic to a new version and watching its SLIs before proceeding.

Layer 4 — The tier-one service, where the patterns live

This layer is drawn as two rows: the request-handling logic on top, and the guards that wrap every outbound call below it.

Idempotency layer

Every write passes through here first. The request's idempotency key is looked up; if it has been seen before, the stored response is returned immediately and no work is repeated. It is drawn before the business logic because that is precisely where it must sit — duplicate detection is worthless if the duplicate has already been executed.

Business logic and orchestrator

The actual work of the service, and the place where graceful degradation is decided. The orchestrator knows which of its calls are critical and which are optional, and it assembles the best response it can from whatever succeeded. A response missing an optional section is a success; a response missing a critical section is an honest, fast failure.

The outbound call guards

The red band containing four numbered boxes is the standard wrapper applied to every call the service makes to anything external. The numbers indicate the order in which they are evaluated, which is deliberate:

35. Timeout — the call can never take longer than its cap, nor longer than the remaining deadline.
36. Retry — applied only to safe, idempotent operations, with exponential backoff, jitter and a retry budget.
37. Circuit breaker — tracks the recent failure rate and, once it trips, prevents calls from being attempted at all.
38. Bulkhead — caps how many calls to this particular dependency can be in flight simultaneously.

At execution time these are nested rather than sequential. The bulkhead is checked first (is there a slot?), then the breaker (is this dependency even worth calling?), then the timed call itself, and the retry logic wraps the whole attempt. Section 5.6 explains why that order is the correct one.

Layer 5 — Downstream dependencies, split by criticality

The colour split in this row carries the whole meaning of graceful degradation.

Dependency	Class	What happens when it fails
Payments	Critical	There is no honest fallback for taking money. The request fails quickly with a clear message stating that nothing was charged. Failing fast and truthfully is the correct behaviour here.
Inventory / core database	Critical	Reads fall back to a replica, which keeps browsing alive. Writes fall back to being queued. A total loss of both means the service enters read-only mode.
Recommendations	Optional	A generic best-sellers list is substituted. Almost no user notices.
Reviews and badges	Optional	The section is omitted entirely and the rest of the page renders normally.

The label ‘graceful degradation applies here’ sits over the two optional dependencies because that is where degradation earns its value: those failures are converted into a slightly reduced page rather than an outage.

Layer 6 — The fallback and asynchronous safety net

- **Cache (Redis).** Holds recently computed answers. When the source of truth is unavailable, a value that is sixty seconds old is served instead of an error. It is filled continuously during normal operation so that it is already warm when it is needed.
- **Static defaults.** Safe values compiled into the service itself — a standard shipping estimate, a default product ranking. They depend on nothing, so they cannot fail. They are the last line before the service has to admit defeat.
- **Queue (Kafka or SQS).** The asynchronous fallback for writes. The request is accepted, durably recorded and processed once the dependency recovers. This converts an outage into a delay, which is a far smaller loss.
- **Read replica.** A second copy of the database used for reads when the primary is unavailable. Data may lag by seconds, which is almost always acceptable for browsing and almost never acceptable for a balance check — so this fallback is enabled per operation, not globally.

5.5 The measurement and control plane (the right-hand column)

This column never touches user traffic. It observes it, converts observations into decisions, and feeds those decisions back into engineering priorities. The dashed arrows crossing from the request path into it are the telemetry taps; the long dashed arrow returning from the bottom is the feedback path.

Stage	What happens	Why it is placed there
Telemetry collection	Every hop emits metrics, structured logs and a shared trace ID	Measurement is taken at the edge, where the user's real experience is visible, not inside the service
SLIs	Raw telemetry is turned into ratios: success rate, latency distribution, freshness	These are the only numbers that describe user experience rather than machine state
SLOs	The targets are applied: 99.99% success, p99 under 300 ms, over a rolling 28-day window	Turns a measurement into a commitment that can be met or missed
Error budget	The gap between target and perfection, tracked as a remaining balance	Makes reliability a quantity that can be spent, planned and traded off
Burn-rate alerts	Fast burn pages a human immediately; slow burn opens a ticket	Catches problems long before the budget is gone, without waking anyone unnecessarily
Incident response	On-call engineer, runbook, mitigate first and diagnose second, blameless review afterwards	At nine seconds of allowed downtime per day, humans mitigate and learn; they do not detect

Stage	What happens	Why it is placed there
Feedback into engineering	Timeouts retuned, thresholds adjusted, pool sizes changed, new fallbacks added, game days re-run	Closes the loop: every incident makes the next one smaller

5.6 Why the guards are applied in that specific order

The ordering of the four outbound guards is not arbitrary. Getting it wrong silently removes most of their value.

Order	Guard	Why it must be at this position
1st checked	Bulkhead	If there is no free slot for this dependency, nothing else should even be attempted. Checking it first means an overwhelmed dependency cannot consume resources merely by being asked.
2nd checked	Circuit breaker	If the dependency is known to be down, the call must be skipped before a timeout is spent waiting for it. A breaker placed after the timeout would save nothing at all.
3rd	Timeout	Wraps the actual attempt, so that this one attempt cannot hang.
Outermost	Retry	Retries wrap the whole attempt, so each retry gets its own timeout, its own breaker check and its own bulkhead slot. A retry that bypassed the breaker would defeat the breaker's purpose entirely.

A frequent and expensive mistake: Placing retries inside the timeout rather than outside it. If the total timeout is 250 ms and the retry logic sits within it, the retries never actually get a chance to run. If instead each retry has its own 250 ms timeout with no overall cap, three attempts plus backoff can take well over a second — blowing through a deadline the caller has already given up on.

6. The Workflow, End to End

Chapter 5 described the structure. This chapter describes the motion: what actually happens, in order, when a request arrives — first when everything is healthy, then when things break, and then what the humans do.

6.1 The four workflows that make up the system

Workflow	Trigger	Timescale	Covered in
The request workflow	A user or client sends a request	Milliseconds	Sections 6.2–6.4
The degradation workflow	A dependency starts failing	Seconds to minutes	Section 6.5
The incident workflow	A burn-rate alert fires	Minutes to hours	Section 7.4
The governance workflow	Weekly and monthly review cycles	Days to weeks	Sections 7.2–7.3

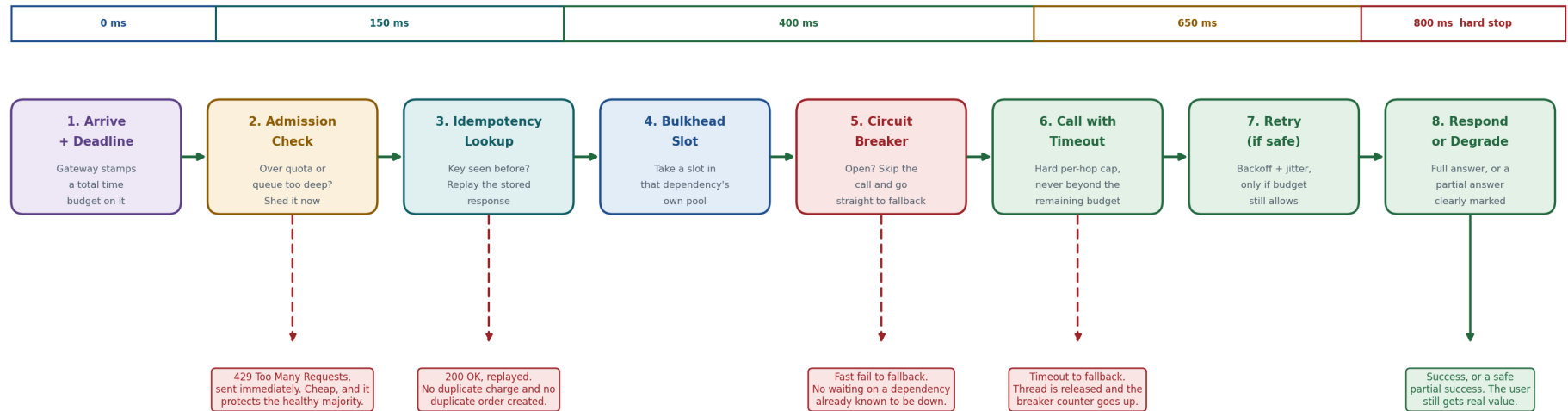
6.2 Figure 2 — The lifecycle of a single request

Follow one request from left to right. At each gate it either continues along the green path or is stopped early, in a controlled way, along a red path. The crucial property is that there is no path where the request simply hangs: every outcome is reached deliberately and within the time budget.

Figure 2 - Lifecycle of a Single Request Through the Guards

Follow one request from left to right. At each gate it either continues (green) or is stopped early in a controlled way (red). Nothing is ever left hanging.

Time budget consumed along the way (the deadline travels with the request)



Rule of thumb: the sum of every hop's timeout plus its retries must stay under the caller's deadline. Otherwise the caller gives up while the callee is still working, burning capacity to produce a reply that nobody will ever read.

Figure 2 — The eight gates every request passes through, the early exits available at each, and the time budget being consumed along the way.

6.3 The eight steps in detail

Step 1 — Arrival and deadline assignment

The request reaches the gateway. It is authenticated, and it is stamped with a total time budget, say 800 ms, along with a trace ID that will follow it through every hop. From this moment the deadline is the supreme authority in the request's life: no later stage may exceed it, and any stage that finds the remaining budget too small to be useful abandons its work immediately rather than doing it pointlessly.

Step 2 — Admission control

Before any real work begins, the service asks whether it should accept this request at all. It checks the caller's quota, the current concurrency against its adaptive limit, and — the most useful signal — how long requests are currently waiting in the queue.

If the service is over capacity, the request is rejected here with an HTTP 429 and a Retry-After header. This costs microseconds. The rejection is deliberate and priority-aware: background and batch traffic is dropped first, then retried traffic, then anonymous browsing, while the critical journey is protected until last. Requests whose deadline has already passed while queuing are discarded without being processed, since no client is still listening for them.

Step 3 — Idempotency lookup

For any request that changes state, the idempotency key is checked before anything else happens. Three outcomes are possible:

- The key is new: it is recorded as in-progress, and the request proceeds.
- The key exists and is complete: the stored response is returned immediately. No work is repeated, and the client cannot distinguish this from the original response.
- The key exists and is still in progress: the client is told to retry shortly, or the request waits briefly for the first execution to finish. A second execution is never started.

Step 4 — Acquiring a bulkhead slot

For each downstream call, the service must first obtain a slot in that dependency's own dedicated pool. If the pool is full, the call is not queued indefinitely — it is rejected immediately and the request moves to its fallback. A full pool means that dependency is already struggling, and adding a queued call would only add latency to a request that is likely to fail anyway.

Step 5 — Consulting the circuit breaker

With a slot in hand, the service checks the breaker for that dependency. If it is open, the call is skipped entirely and the fallback runs at once. This saves the entire timeout period — the single largest latency saving available during a dependency outage — and it gives the failing dependency the quiet it needs in order to recover.

Step 6 — Making the call, under a timeout

The call is finally made. Its effective timeout is the smaller of the configured per-attempt timeout and the budget still remaining from step 1. If the call returns in time, the result is used. If it times out, the thread is

released immediately, the failure is recorded against the breaker's rolling window, and control moves to step 7.

Step 7 — Retrying, but only when it is both safe and useful

A retry happens only when all of the following are true: the operation is idempotent or carries an idempotency key; the error was of a retryable kind; the attempt limit has not been reached; the retry budget for that dependency has not been exhausted; and enough of the deadline remains to complete another attempt. If all five hold, the service waits for a backoff period with jitter applied and tries again. If any one of them fails, it proceeds to the fallback without delay.

Step 8 — Responding, or degrading

The orchestrator assembles whatever it has. If every critical call succeeded, the user receives a complete response. If an optional call failed, the user receives a response with that section replaced by a fallback or omitted, and a metric is emitted recording that the response was degraded. If a critical call failed, the user receives a fast, clear error explaining what did not happen — for example, that no payment was taken.

6.4 The timeout budget arithmetic

This is the calculation that makes the whole chain coherent. Working from an 800 ms promise to the user:

Stage	Budget consumed	Remaining	Notes
Deadline assigned at the gateway	—	800 ms	The promise made to the user
Authentication, admission and idempotency check	~20 ms	780 ms	Cheap by design, because every request pays it
Critical call 1 (inventory), one attempt	up to 150 ms	630 ms	Timeout set from the dependency's p99.9
Critical call 2 (payments), attempt plus one retry	up to 320 ms	310 ms	250 ms attempt, up to 70 ms of jittered backoff
Optional call (recommendations)	up to 80 ms	230 ms	A deliberately tight timeout: never let a nice-to-have consume much budget
Assembling and serialising the response	~30 ms	200 ms	Held in reserve for network transit and variance

The rule to remember: Every hop's total time — attempts plus backoff waits — must fit inside the caller's remaining deadline, with margin to spare. Optional dependencies get the tightest timeouts of all, because their value is low and the budget they consume is taken directly from the critical path.

6.5 Worked example: a dependency fails, minute by minute

This is the degradation workflow in action. The recommendation service, an optional dependency, begins failing during a normal weekday morning.

Time	What happens	Which mechanism acted	User impact
09:14:00	The recommendation service starts returning timeouts after a bad deployment on its side	—	None yet
09:14:00.25	The first calls hit their 80 ms timeout. Threads are released rather than held.	Timeout	80 ms of extra latency on affected pages
09:14:01	Concurrency to that dependency reaches its bulkhead limit of 20. Further calls are refused instantly.	Bulkhead	None — checkout and search are untouched, because they use different pools
09:14:03	Twenty failures are recorded in the rolling window and the failure rate exceeds 50%. The breaker opens.	Circuit breaker	Extra latency disappears; the page now renders at normal speed
09:14:03	Every product page now serves the generic best-sellers list from cache instead of personalised results	Fallback + graceful degradation	Recommendations are generic rather than personal. Most users do not notice.
09:14:05	The degradation metric rises and the dashboard shows the breaker as open	Telemetry	None
09:14:30	The breaker enters half-open and allows three trial calls. All three fail. It returns to open.	Circuit breaker	None
09:18:00	A slow-burn alert opens a ticket. No page is sent, because the error budget is barely moving — the user-visible success rate is still above 99.99%.	Burn-rate alerting	None
09:31:00	The recommendation team rolls back their deployment	Human response	None
09:31:30	The next half-open trial succeeds. The breaker closes and personalised results resume automatically.	Circuit breaker	Full service restored without any action on this service's side

The point of that timeline: A complete outage of a downstream dependency, lasting seventeen minutes, produced no user-visible outage, no page in the middle of the night, and almost no error-budget consumption. Before this work, the same event would have exhausted several days of budget and very likely taken checkout down with it. That difference is the whole project in a single example.

6.6 Worked example: the double payment that does not happen

This is the idempotency workflow, which is the one most often skipped and the one whose absence causes the most damaging failures.

Step	What happens
1	The customer taps Pay. The app generates an idempotency key, a UUID, and sends it with the request.
2	The service records the key as in-progress, charges the card successfully, and stores the completed status and the exact response body against the key, in the same transaction as the charge itself.
3	The response is lost on the way back — the customer's train enters a tunnel. The app sees a timeout and has no idea whether the payment succeeded.
4	The app retries, after a jittered backoff, using the same idempotency key. This is safe precisely because the key is the same.
5	The service finds the key already present and marked complete. It performs no work and returns the stored response.
6	The customer sees a successful payment. Exactly one charge exists. The customer never learns that anything went wrong.

Without step 2 storing the response transactionally alongside the charge, step 5 could return a response that disagrees with what actually happened to the money. Without the key being generated per logical operation rather than per network attempt, step 4 would create a second charge. Both details are small, and both are the entire point.

7. The Governance Workflow and Blast Radius

Figure 3 shows the two things that turn the machinery in Chapters 4 to 6 into a lasting result. On the left is the governance loop — the cycle that keeps the targets honest and decides how the team spends its time. On the right is a concrete demonstration of blast radius: the same dependency failure, with and without the protective patterns in place.

Figure 3 - The Error-Budget Loop and How Blast Radius Is Contained

Left: the governance cycle that decides whether the team ships or fixes. Right: what a failing dependency can and cannot take down.

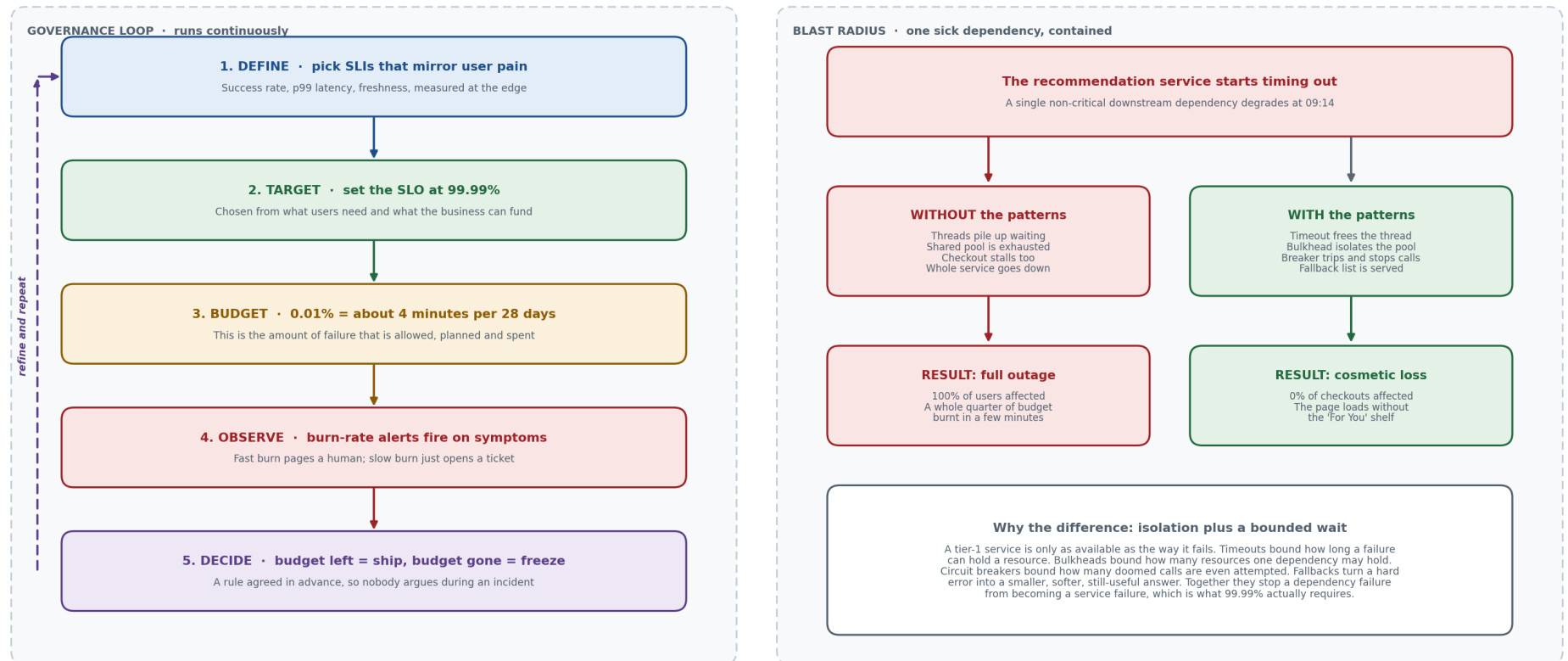


Figure 3 — Left: the five-stage error-budget loop. Right: how the same failure produces either a full outage or a cosmetic loss, depending entirely on whether the patterns are present.

7.2 The five stages of the governance loop

The left-hand side of Figure 3 is a cycle, not a checklist. It runs continuously, and each pass through it refines the one before.

Stage	What happens	Who is involved	Output
1. Define	Choose SLIs that reflect what users actually feel, and write down exactly what counts as a good event and a valid event	Service team plus product owner	A written SLI specification per service
2. Target	Set the SLO from what users need and what the business can afford to fund	Service team, product, leadership	An agreed target and window, for example 99.99% over 28 days
3. Budget	Convert the gap into an error budget and agree the policy for what happens at each level of depletion	Same group, agreed in advance and in writing	A budget policy nobody has to negotiate mid-incident
4. Observe	Track burn rate continuously; alert on symptoms with fast and slow burn thresholds	Automated, with on-call engineers	Alerts that are trusted because they are always real
5. Decide	Apply the agreed policy: healthy budget means ship, depleted budget means fix reliability	Service team and product owner	A release decision that is evidence-based rather than political

The dashed arrow returning from stage 5 to stage 1 matters. Targets that are never revisited go stale: traffic patterns shift, the product changes, and an SLI that was meaningful a year ago may now be measuring something nobody cares about. Reviewing the definitions themselves is part of the loop, not an exception to it.

7.3 The operating cadence

The loop is anchored to a fixed rhythm, so that it happens whether or not anyone remembers to schedule it.

Frequency	Activity	Purpose
Continuous	Automated SLI computation, burn-rate evaluation, dashboards	Detection in seconds, since nine seconds a day leaves no room for human detection
Daily	On-call handover reviewing budget consumption and any degradation events from the previous day	Catches slow, quiet problems before they compound
Weekly	Reliability review: budget remaining, top causes of consumption, progress on reliability work	Keeps reliability visible next to feature work rather than beneath it

Frequency	Activity	Purpose
Per incident	Blameless postmortem within five working days, with owned and dated actions	Turns each failure into a permanent reduction in future risk
Monthly	Review of timeouts, thresholds, pool sizes and quotas against current traffic	Configuration ages; this is when it is corrected
Quarterly	Game days and failure injection; re-examination of whether the SLOs are still the right ones	Proves the fallbacks work, and keeps the targets honest

7.4 The incident workflow

When a fast-burn alert fires, the sequence is fixed and practised.

39. Acknowledge. The on-call engineer confirms receipt, so that escalation timers are accurate.
40. Assess scope using the SLO dashboards: which SLI is affected, how fast the budget is burning, which customers or regions are involved.
41. Mitigate before diagnosing. Roll back the recent deployment, shift traffic to a healthy region, force a degradation level, or open a breaker manually. Stopping the bleeding always comes before understanding the cause — with a four-nines target there is simply no time to do both in the other order.
42. Communicate. Update the status page and internal channels with what is known, what is being done, and when the next update will come.
43. Verify recovery against the SLIs, not against a feeling. The incident ends when the numbers say it has ended.
44. Record the cost: how much of the error budget the incident consumed.
45. Run a blameless postmortem within five working days, producing dated, owned actions. Blameless is not a nicety — people who fear blame hide the information that prevents the next incident.
46. Feed the actions back into the design: adjust a timeout, resize a pool, add a missing fallback, tighten a quota, add a test. This is the arrow that closes the loop in Figure 1.

7.5 The release workflow

Because most incidents are caused by change, the release process is itself a reliability control, gated by the error budget.

Phase	What happens	Gate before proceeding
Pre-release check	Confirm the error budget has enough headroom for the risk of this change	If the budget is below 25%, only reliability fixes proceed

Phase	What happens	Gate before proceeding
Canary	Route 1% of traffic to the new version	SLIs on the canary must match the baseline for the full bake period
Progressive rollout	5%, then 25%, then 50%, then 100%, with a bake period at each step	Automatic rollback if the error rate or latency SLI degrades
Post-release watch	Heightened monitoring for a defined period afterwards	Budget consumption attributed to the release, so its true cost is known

7.6 Reading the right-hand side of Figure 3: blast radius

The right side of the diagram shows one event — the recommendation service timing out — following two different paths.

Without the patterns, threads waiting on the failing dependency accumulate in a shared pool. The pool is exhausted. Checkout, which does not use recommendations at all, can no longer be served because it needs threads from that same pool. The result is a full outage affecting every user, consuming an entire quarter's error budget in a few minutes, over a decorative feature.

With the patterns, each mechanism bounds one dimension of the damage. The timeout frees the thread after 80 ms. The bulkhead prevents that dependency from ever holding more than twenty slots. The circuit breaker stops the calls entirely once failure is established. The fallback supplies a generic list. The result is that no checkout is affected at all, and the only visible change is a slightly less personalised shelf on the product page.

The whole project, compressed into one sentence: The failure was identical in both cases. Only the design of the failure path was different — and that difference is the entire distance between a full outage and something most users never notice.

8. How the Components Work Together

The individual patterns are each fairly simple. Their value comes from the combinations, and several of them are actively dangerous when used without their partner.

Pairing	Why they belong together
Timeout + retry	A retry is only meaningful because the timeout ended the first attempt at a known moment. Without timeouts, there is nothing to retry from.
Retry + idempotency	This is the most important pairing in the document. Retrying a non-idempotent write is a mechanism for duplicating customer orders and payments. Never deploy one without the other.
Retry + jitter + retry budget	Retries without jitter create synchronised waves; retries without a budget multiply load precisely when a dependency is weakest. Both safeguards are needed for retries to help rather than harm.
Timeout + circuit breaker	The timeout bounds one bad call; the breaker stops you making thousands of them. Timeouts alone still pay the full waiting cost on every single request during an outage.
Circuit breaker + fallback	An open breaker fails instantly. Without a fallback, all that achieves is failing faster. The fallback is what converts fast failure into continued service.
Bulkhead + shedding	The bulkhead stops one dependency consuming everything; shedding stops total demand consuming everything. They protect against different sources of the same exhaustion.
Degradation + observability	A service can run degraded for weeks while its success-rate dashboard reads 100%. Without a metric on every fallback, degradation becomes an invisible, permanent loss of quality.
Error budget + everything else	The budget is what tells you whether all of this machinery is set correctly, and what justifies the engineering time to keep improving it.

The dependency chain, stated as a rule: Idempotency makes retries safe. Timeouts make failures fast. Breakers make repeated failures cheap. Bulkheads make failures local. Shedding makes overload survivable. Fallbacks make failures invisible. Error budgets make the whole thing measurable and self-correcting.

9. Reference Configuration

These are sensible starting values, not universal truths. Every one of them should be replaced by a number derived from your own measured latency distributions and load tests. They are included so that the earlier chapters have something concrete to point at.

Setting	Starting value	How to derive the real value
Request deadline (end to end)	800 ms	From the product requirement for perceived responsiveness
Per-attempt timeout	The dependency's p99.9 latency plus roughly 20%	From the dependency's own latency histogram, reviewed monthly
Connection timeout	100 ms	Should be well under the read timeout; unreachable hosts must be detected quickly
Retry attempts	2 additional attempts at most	Rarely worth more; beyond three, a fallback is almost always better
Retry backoff base	100 ms, doubling, capped at 2 s	From the dependency's typical recovery time
Jitter	Full jitter: a random wait between zero and the computed backoff	Use full jitter unless there is a specific reason not to
Retry budget	Retries capped at 10% of requests to that dependency	Enforced with a token bucket; this is the main defence against retry storms
Breaker failure threshold	50% failures over a 20-call minimum in a rolling window	Tune so that normal noise never trips it and a real outage always does
Breaker open duration	30 seconds	Long enough for a restart or failover, short enough to recover promptly
Breaker half-open trials	3 calls	Enough evidence of recovery without flooding a fragile dependency
Bulkhead pool size	Expected concurrency plus 50% headroom	Concurrency = throughput × latency (Little's Law), measured under load
Shedding signal	Queue wait time above 100 ms, or an adaptive concurrency limit	Prefer measured latency over a fixed requests-per-second cap
Idempotency key retention	24 hours	Comfortably longer than any client will keep retrying
Cache fallback staleness	Up to 60 seconds for prices, up to 24 hours for recommendations	From how much staleness the business can genuinely tolerate per data type
SLO window	Rolling 28 days	Rolling avoids an artificial budget reset at a month boundary

10. Common Mistakes

Each of these has caused a major public outage at some well-known company. They are listed because the patterns in this document are only as good as the details of their implementation.

Mistake	What goes wrong	The fix
No timeout, or an infinite default	One slow dependency consumes every thread and the whole service stops responding	Set an explicit timeout on every external call, and audit client library defaults — many default to no timeout at all
Retrying at every layer	Three layers each retrying three times turns one user action into twenty-seven calls to a dependency that is already failing	Choose one layer to own retries for each hop; every other layer passes the failure through
Retries without jitter	Every client retries at the same instant, producing synchronised waves that repeatedly knock over a recovering service	Apply full jitter to every backoff calculation
Retrying non-idempotent writes	Duplicate charges and duplicate orders — a data-integrity failure, which is worse than downtime	Require an idempotency key on every write before enabling any retry
Inner timeouts larger than the outer deadline	The caller gives up while the callee is still working; capacity is burned producing answers nobody will read	Propagate the deadline and always take the smaller of the configured timeout and the time remaining
Breaker thresholds too sensitive	The breaker flaps open and closed on normal noise, causing more disruption than the problem it was protecting against	Require a minimum call volume in the window, and use a percentage rather than an absolute count
A fallback that depends on the failed component	The fallback fails for the same reason as the primary path, at the worst possible moment	Fallbacks must use a completely separate path, be pre-warmed, and never call the broken dependency
Untested fallback paths	Code that has never run in production executes for the first time during your worst incident	Exercise every fallback in scheduled game days
Degradation with no metric	The service runs degraded for weeks while every dashboard reads 100% success	Emit a metric on every fallback and every degradation, and alert on the sustained rate
Alerting on causes instead of symptoms	Engineers are woken for high CPU on a service that is serving every request correctly, and stop trusting the pager	Alert on user-visible symptoms tied to error-budget burn rate
An SLO with no consequence	It becomes a decorative dashboard that nobody acts on	Write the budget policy in advance and actually enforce the freeze

Mistake	What goes wrong	The fix
Measuring the SLI inside the service	Requests that never arrived are invisible, so the numbers look better than the user's experience	Measure at the edge, or from the client, or with external probes
Queueing instead of shedding under overload	Everything slows until everything times out, and useful throughput collapses to near zero	Shed early, shed cheaply, and shed by priority
No per-tenant limits	One customer's runaway script consumes the capacity of every other customer	Apply quotas per tenant, and consider shuffle sharding for stronger isolation

11. Summary

11.1 The project in sixty seconds

If you have to explain this out loud: We ran services where an outage directly costs revenue, so the target was 99.99% — which allows under nine seconds of failure a day, far too little for a human to detect and fix anything. That constraint drove the whole design. First we defined reliability precisely: SLIs measured at the edge for success rate and latency, SLOs of 99.99% over a rolling 28-day window, and the remainder as an error budget with an agreed policy — healthy budget means ship, exhausted budget means stop and fix. Then we made the service fail small. Every outbound call got a timeout bounded by an end-to-end deadline; retries only for idempotent operations, with exponential backoff, full jitter and a retry budget; circuit breakers so we stop calling what is already broken; bulkheads so no dependency can consume more than its own share of threads; request shedding so overload rejects a few requests fast instead of failing everything slowly; idempotency keys so a retried payment can never charge twice; and a fallback for every dependency, classified as critical or optional in advance. The result is that a downstream failure now costs us a missing recommendations shelf instead of a checkout outage — the blast radius shrank from the whole service to one section of one page.

11.2 The five ideas worth remembering

47. A tier-one service is only as available as the way it fails. You cannot stop dependencies from breaking; you can decide in advance how much breaks with them.
48. Slow is more dangerous than down. A failure returns resources immediately; slowness quietly consumes all of them while looking healthy.
49. Failing fast, honestly and partially beats failing slowly and completely — for users, for the error budget and for the recovery of the dependency.
50. Retries are a safety mechanism and an amplification mechanism at the same time. Idempotency, jitter and a retry budget are what decide which one you get.
51. What is not measured at the edge, and not tested in production, does not really exist — including your fallbacks.

11.3 Implementation checklist

A condensed version of the whole document, in the order it is worth doing.

#	Step	Done when
1	Classify services into tiers and dependencies into critical, important and optional	Every dependency has a written class and a documented behaviour on failure
2	Define SLIs measured at the edge	Good and valid events are defined unambiguously, per service

#	Step	Done when
3	Set SLOs and windows, and publish the error-budget policy	The policy is agreed in writing before any incident
4	Build the SLO dashboard and multi-window burn-rate alerts	Fast and slow burn alerts exist, and every page is trusted
5	Set an explicit timeout on every external call and propagate deadlines	No call anywhere can hang; inner budgets fit inside outer ones
6	Add idempotency keys to every write path	A repeated key provably returns the stored response
7	Add retries with exponential backoff, full jitter and a retry budget	Only idempotent, retryable failures are retried, at exactly one layer
8	Add circuit breakers per dependency and per operation	Breaker state is visible on the dashboard
9	Add bulkheads sized from measured concurrency	Pool utilisation is a metric, and no dependency shares a pool
10	Add priority-aware request shedding at the edge, gateway and service	Overload produces fast rejections, not a growing queue
11	Implement a fallback for every dependency and define degradation levels	Every fallback emits a metric when used
12	Run game days and load tests, and feed the results back	Every fallback path has executed in production under supervision

End of document.